LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc đến Thầy Trần Hoàng Linh đã tận tình hướng dẫn em trong suốt quá trình thực hiện đồ án 2 tại Trường Đại học Bách Khoa, Đại học Quốc gia Thành phố Hồ Chí Minh.

Trong quá trình thực hiện, em nhận thấy rằng sự hỗ trợ của Thầy không chỉ giúp em hoàn thành đồ án mà còn mang lại cho em những bài học ý nghĩa về sự kiên trì và tinh thần trách nhiệm. Em rất mong những kinh nghiệm này sẽ tiếp tục đồng hành cùng em trong các dự án và thử thách sắp tới.

Một lần nữa, em xin chân thành cảm ơn Thầy Trần Hoàng Linh vì tất cả sự hướng dẫn và động viên vô giá trong hành trình này. Em hy vọng sẽ tiếp tục nhận được sự dìu dắt từ Thầy trong những chặng đường phía trước.

Em xin chân thành cảm ơn.

Thành phố Hồ Chí Minh, ngày 30 tháng 12 năm 2025

Sinh viên

Lê Tuấn Khanh

TÓM TẮT ĐỒ ÁN

Đồ án này tập trung vào việc nghiên cứu, thiết kế và hiện thực hóa giao thức bus TileLink Uncached Lightweight (TL-UL), một chuẩn giao tiếp on-chip mã nguồn mở thường được sử dụng trong các hệ thống SoC dựa trên kiến trúc RISC-V. Dự án được thực hiện nhằm giải quyết bài toán kết nối giữa bộ xử lý (Master) và các thiết bị ngoại vi hoặc bộ nhớ (Slave) một cách hiệu quả, tiết kiệm tài nguyên phần cứng và tuân thủ các chuẩn công nghiệp.

Nội dung chính bao gồm việc xây dựng các thành phần cốt lõi của giao thức: Host Adapter (giao tiếp phía Master), Device Adapter (giao tiếp phía Slave), và Crossbar để định tuyến dữ liệu. Các module được thiết kế bằng ngôn ngữ SystemVerilog, tuân thủ nghiêm ngặt quy tắc bắt tay (handshake) Valid/Ready để đảm bảo không xảy ra hiện tượng tắc nghẽn (deadlock) hay mất mát dữ liệu trong quá trình truyền tải.

Các thiết kế đã được kiểm tra kỹ lưỡng thông qua môi trường Testbench với các kịch bản kiểm thử ngẫu nhiên (Randomized Testing) và kiểm thử các trường hợp biên như nghẽn mạch (backpressure). Kết quả mô phỏng dạng sóng (Waveform) cho thấy hệ thống hoạt động chính xác với các lệnh Get (Đọc) và Put (Ghi), đáp ứng đúng đặc tả kỹ thuật của SiFive. Đồ án cũng đề xuất phương án tích hợp bus này vào bộ xử lý RISC-V đã thiết kế trước đó để triển khai trên FPGA.

MỤC LỤC

I. GIỚI THIỆU 5
1. Tổng quan 5
2. Nhiệm vụ đề tài 5
II. Cở sở lý thuyết giao thức TileLink Uncached Lightweight 5
1. Giới thiệu về TileLink 5
2. TileLink Uncached Lightweight (TL-UL) 6
2.1 Tổng quan về TL-UL 6
2.2. Các loại hoạt động trong TL-UL 7
2.3. Cấu trúc kênh trong TL-UL 7
2.3.1. Kênh A – Request Channel 9
2.3.2. Kênh D – Response Channel 10
2.4. Ordering và Outstanding Transaction 10
III. Phương pháp thiết kế 11
1. Tổng quan thiết kế 11
1.1. Alignment Check – Kiểm tra địa chỉ 12
1.2. Request/Response FIFO 12
1.3. FSM Controller 13
1.4. Outstanding Transaction Counter 14
1.5. Response Handler – Trả kết quả cho LSU 15
2. Thiết kế TL-UL Crossbar 16
2.1. Address Decoder 17
2.2. Request DEMUX – Định tuyến Channel A 18
2.3. Source ID Table 18
2.4. Error Responder FSM 18
2.5. Response Arbiter và Round-Robin Logic 19
2.6. Channel D Routing về Master 19
3. Thiết kế các Slave Adapter 19
3.1. Thiết kế TL-UL Memory Adapter 19
3.2. TL-UL Peripheral Adapter 20
4. Thiết kế UART và TL-UL UART Bridge 20
4.1. Thiết kế UART Transmitter (TX) 20
4.2. Thiết kế UART Receiver (RX) 20
4.3. Thiết kế TL-UL UART Bridge 21

IV. Kiểm tra và xác minh thiết kế 22
1. Mục tiêu kiểm tra và xác minh 22
2. Kiểm tra và xác minh TL-UL Crossbar và các Slave Adapter 22
3. Kiểm tra xử lý backpressure và flow control 25
4. Kiểm tra và xác minh UART và TL-UL UART Bridge 26
5. Kiểm tra TL-UL bus khi tích hợp vào core RISC-V 26
V. Đánh giá hiệu năng Bus (Throughput) 28
VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 31
VII. TÀI LIỆU THAM KHẢO 33

DANH SÁCH HÌNH ẢNH

HÌNH 1. SƠ ĐỒ KHỐI TỔNG QUÁT THIẾT KẾ 11
HÌNH 2. SƠ ĐỒ KHỐI HOST ADAPTER..... 12
HÌNH 3. SƠ ĐỒ KHỐI REQUEST/RESPONSE FIFO 13
HÌNH 4. THIẾT KẾ FSM 14
HÌNH 5. SƠ ĐỒ KHỐI BỘ OUTSTANDING COUNTER 15
HÌNH 6. SƠ ĐỒ KHỐI BỘ RESPONSE HANDLER 15
HÌNH 7. SƠ ĐỒ KHỐI TỔNG QUÁT TL-UL CROSSBAR..... 16
HÌNH 8. MAPPING ĐỊA CHỈ CÁC SLAVE..... 17
HÌNH 9. WAVEFORM KIỂM CHỨNG TÁC VỤ CƠ BẢN CỦA TL-UL..... 23
HÌNH 10. WAVEFORM KIỂM TRA BYTE ACCESS 23
HÌNH 11. WAVEFORM KIỂM TRA KHẢ NĂNG GHI THEO TỪNG BYTE 24
HÌNH 12. WAVEFORM KIỂM TRA KHẢ NĂNG GHI HALFWORD..... 24
HÌNH 13. WAVEFORM KIỂM CHỨNG KHẢ NĂNG GHI VÀO LED VÀ ĐỌC INPUT TỪ SWITCH 25
HÌNH 14. WAVEFORM KIỂM TRA TRƯỜNG HỢP ACCESS ĐỊA CHỈ KHÔNG ĐƯỢC MAP..... 25
HÌNH 15. WAVEFORM KIỂM TRA SALVE BACKPRESSURE 25
HÌNH 16. WAVEFORM KIỂM TRA MASTER BACKPRESSURE..... 26
HÌNH 17. WAVEFORM KIỂM TRA KHẢ NĂNG TRUYỀN NHẬN CỦA UART .. 26
HÌNH 18. WAVEFORM KIỂM TRA KHẢ NĂNG TRUYỀN NHẬN KHI UART ĐƯỢC GÁN VỚI TL-UL 26
HÌNH 19. KẾT QUẢ KIỂM TRA UART TRUYỀN NHẬN TRONG SOC..... 28
HÌNH 20. TÓM TẮT TÀI NGUYÊN SỬ DỤNG TRÊN KIT DE10..... 30
HÌNH 21. TẦN SỐ TỐI ĐA CỦA THIẾT KẾ..... 31

DANH SÁCH BẢNG BIỂU

BẢNG 1. TỔNG QUAN CÁC KÊNH TRONG TILELINK 7
BẢNG 2. CÁC LOẠI TÍN HIỆU TRONG TILELINK 8
BẢNG 3. CÁC THAM SỐ TRONG CÁC TÍN HIỆU 8
BẢNG 4. CÁC TÍN HIỆU KÊNH A 9
BẢNG 5. CÁC TÍN HIỆU KÊNH D 10
BẢNG 6. KẾT QUẢ TÍNH TOÁN THROUGHPUT CỦA BUS 29

I. GIỚI THIỆU

1. Tổng quan

Trong thiết kế Vi mạch (System-on-Chip - SoC), việc kết nối các khối chức năng (IP Cores) đóng vai trò quyết định đến hiệu năng toàn hệ thống. TileLink là một giao thức bus mã nguồn mở, miễn phí, được thiết kế bởi SiFive và cộng đồng RISC-V để khắc phục các nhược điểm về độ trễ và sự phức tạp của các chuẩn cũ như AXI hay AHB. Đồ án này tập trung vào phiên bản TileLink Uncached Lightweight, phiên bản rút gọn dành cho các thiết bị ngoại vi công suất thấp và không yêu cầu tính nhất quán bộ nhớ đệm (cache coherency).

2. Nhiệm vụ đề tài

Đề tài được chia thành các nhiệm vụ cụ thể, với yêu cầu, kết quả cần đạt và giới hạn rõ ràng. Dưới đây là các nội dung chính cùng với cách tiếp cận và ý tưởng thực hiện:

- Nội dung 1: Nghiên cứu đặc tả kỹ thuật (Specification) của TileLink phiên bản 1.9.3, tập trung phân tích hoạt động của lớp TL-UL.
- Nội dung 2: Thiết kế và hiện thực hóa các module chuyển đổi giao thức (Adapter) cho phía Master và Slave sử dụng ngôn ngữ mô tả phần cứng SystemVerilog.
- Nội dung 3: Xây dựng bộ Interconnect 1-to-N (1 Master kết nối nhiều Slave) bao gồm logic giải mã địa chỉ và bộ dồn kênh.
- Nội dung 4: Xây dựng môi trường kiểm thử (Testbench) để xác minh tính đúng đắn của giao thức, bao gồm cả các trường hợp gây nghẽn bus (Backpressure).

II. Cở sở lý thuyết giao thức TileLink Uncached Lightweight

1. Giới thiệu về TileLink

TileLink là một chuẩn liên kết on-chip (chip-scale interconnect) được thiết kế để cung cấp cơ chế truy cập bộ nhớ ánh xạ (memory-mapped) giữa nhiều master và slave trong một hệ thống SoC. TileLink hướng đến việc kết nối các thành phần như CPU đa nhân, bộ tăng tốc (accelerator), bộ điều khiển DMA, bộ nhớ và các ngoại vi, thông qua một giao thức có độ trễ thấp, khả năng mở rộng cao và đảm bảo tính đúng đắn hình thức.

TileLink là một chuẩn mở và miễn phí, được phát triển ban đầu cho kiến trúc RISC-V nhưng không phụ thuộc ISA, do đó có thể áp dụng cho nhiều kiến trúc xử lý khác

nhau. Không giống như các bus truyền thống mang tính tập trung, TileLink được xây dựng trên mô hình kết nối điểm-điểm (point-to-point) và có thể được tổ chức thành các topology linh hoạt, một trong những đặc điểm quan trọng của TileLink là khả năng hỗ trợ bộ nhớ chia sẻ có tính nhất quán (cache-coherent shared memory). Ở mức đầy đủ nhất, TileLink cung cấp cơ chế duy trì tính nhất quán cache tương đương giao thức MESI, cho phép nhiều master có cache cùng truy cập và chia sẻ dữ liệu một cách an toàn. Đồng thời, TileLink cũng có thể thu gọn xuống các thiết bị ngoại vi đơn giản, chỉ yêu cầu các thao tác đọc/ghi cơ bản.

Về mặt kiến trúc, TileLink định nghĩa các giao dịch thông qua các kênh truyền độc lập (channels) với cơ chế bắt tay ready/valid, cho phép pipeline, hoàn thành giao dịch không theo thứ tự (out-of-order) và dễ dàng chèn các tầng thanh ghi để tối ưu timing. Giao thức được thiết kế sao cho đảm bảo không xảy ra deadlock bằng các quy tắc ưu tiên kênh. Để phù hợp với nhiều mức độ phức tạp của hệ thống, TileLink định nghĩa ba mức tuân thủ giao thức: TileLink Uncached Lightweight (TL-UL): hỗ trợ các thao tác đọc/ghi đơn giản, thường dùng cho MMIO và ngoại vi. TileLink Uncached Heavyweight (TL-UH): mở rộng thêm burst, atomic và hint nhưng không có cache coherence. TileLink Cached (TL-C): giao thức đầy đủ, hỗ trợ cache coherence và quản lý quyền truy cập dữ liệu. Nhờ các đặc tính trên, TileLink đặc biệt phù hợp cho các thiết kế SoC hiện đại, các đồ án nghiên cứu CPU/RISC-V, cũng như các hệ thống yêu cầu tính mô-đun, khả năng mở rộng và dễ kiểm chứng hơn so với các chuẩn bus truyền thống. Nó tương tự như chuẩn AXI của ARM nhưng miễn phí và mở.

2. TileLink Uncached Lightweight (TL-UL)

2.1 Tổng quan về TL-UL

TileLink Uncached Lightweight (TL-UL) là mức tuân thủ đơn giản nhất trong các biến thể của giao thức TileLink. TL-UL được thiết kế để phục vụ các truy cập bộ nhớ không cache (uncached memory access), chủ yếu nhắm tới:

- Thanh ghi điều khiển ngoại vi (MMIO)
- Bộ nhớ đơn giản
- Các thiết bị slave có độ phức tạp thấp UART, SPI, Timer, …

TL-UL không hỗ trợ cache coherence, không có cơ chế quản lý quyền truy cập cache, và không yêu cầu các kênh phức tạp như B, C, E. Nhờ đó, TL-UL có chi phí phần cứng

thấp, dễ hiện thực và rất phù hợp cho các đồ án CPU hoặc SoC quy mô nhỏ đến trung bình.

2.2. Các loại hoạt động trong TL-UL

TL-UL chỉ hỗ trợ nhóm lệnh ACCESS, cụ thể gồm hai loại thao tác:

- Get operation (Đọc dữ liệu): Là thao tác dữ liệu tại một địa chỉ xác định, không liên quan đến cache line, slave trả về dữ liệu trực tiếp. Đặc điểm là luôn phải có dữ liệu trả về, không có trường hợp hợp lệ nào mà Get lại nhận AccessAck không có dữ liệu.
- Put operation (Ghi dữ liệu): trong TL-UL, ghi dữ liệu được chia thành 2 dạng:
  - PutFullData: Ghi toàn bộ độ rộng bus tại địa chỉ đích, không sử dụng mask, slave ghi đè trực tiếp. Ưu điểm là logic đơn giản, phù hợp cho thanh ghi điều khiển hoặc ghi nguyên một word.
  - PutPartialData: Ghi dữ liệu đến từng byte, sử dụng tín hiệu *mask* để chỉ định byte hợp lệ, yêu cầu slave phải đọc giá trị, thực hiện merge dữ liệu theo mask và ghi lại kết quả mới. PutPartialData là bắt buộc đối với MMIO vì thanh ghi thường chỉ thay đổi một hoặc vài bit khi ấy phần mềm cần thao tác bit/byte riêng lẻ.

2.3. Cấu trúc kênh trong TL-UL

TL-UL chỉ sử dụng 2 kênh trong số 5 kênh của TileLink.

Bảng 1. Tổng quan các Kênh trong Tilelink

Kênh (Channel)	Hướng (Direction)	Mục đích
A	Master to Slave	Các thông điệp yêu cầu (Request) được gửi đến một địa chỉ cụ thể.
B	Slave to Master	Các thông điệp yêu cầu được gửi đến một khối có bộ nhớ đệm (chỉ TL-C).
C	Master to Slave	Các thông điệp phản hồi (Response) từ một khối có bộ nhớ đệm (chỉ TL-C).
D	Slave to Master	Các thông điệp phản hồi (Response) từ một địa chỉ cụ thể.

E	Master to Slave	Tín hiệu bắt tay cuối cùng cho việc truyền khối bộ nhớ đệm (chỉ TL-C).

Bảng 2. Các loại tín hiệu trong Tilelink

Loại (Type)	Hướng (Direction)	Mô tả
X	Input	Tín hiệu xung nhịp (Clock) hoặc đặt lại (Reset), là đầu vào cho cả hai phía TileLink agent.
C	Channel direction	Các tín hiệu điều khiển (Control), không thay đổi giữa các nhịp (beats) của một lần truyền burst.
D	Channel direction	Các tín hiệu dữ liệu (Data), thay đổi theo từng nhịp (beat).
V	Channel direction	Tín hiệu Valid, chỉ ra rằng các tín hiệu loại C/D đang chứa dữ liệu hợp lệ.
R	Reverse direction	Tín hiệu Ready, chỉ ra rằng tín hiệu V đã được chấp nhận.

Bảng 3. Các tham số trong các tín hiệu

Tham số (Parameter)	Phạm vi (Range)	Mô tả
w	1-4096	Độ rộng của bus dữ liệu tính bằng byte. Phải là lũy thừa của 2.
a	0-128	Độ rộng của mỗi trường địa chỉ tính bằng bit.
z	1-4	Độ rộng của mỗi trường kích thước (size) tính bằng bit.
o	0-64	Số bit cần thiết để phân biệt các nguồn Master (Master sources) trên mỗi liên kết.

i	0-64	Số bit cần thiết để phân biệt các đích Slave (Slave sinks) trên mỗi liên kết.

2.3.1. Kênh A – Request Channel

Kênh A mang các yêu cầu từ Master tới Slave, được thiết kế giúp Master có thể phát nhiều requester liên tiếp mà không cần chờ response trước khi gửi request mới và cho phép khả năng pipeline tốt giảm critical path tăng hiệu suất. Chứa các tín hiệu (signal) quan trọng như sau:

Bảng 4. Các tín hiệu kênh A

Tín hiệu	Loại	Độ rộng	Mô tả
a_opcode	C	3	Mã lệnh (Operation code). Xác định loại thông điệp được truyền tải bởi kênh.
a_param	C	3	Mã tham số. Ý nghĩa phụ thuộc vào a_opcode; chỉ định việc chuyển giao quyền hạn bộ nhớ đệm (caching permissions) hoặc mã phụ (sub-opcode).
a_size	C	z	Logarit cơ số 2 của kích thước hoạt động: 2n bytes.
a_source	C	o	Định danh nguồn Master (Master source identifier) trên mỗi liên kết.
a_address	C	a	Địa chỉ byte đích của hoạt động. Phải được căn chỉnh (aligned) theo a_size.
a_mask	D	w	Chọn làn byte (Byte lane select) cho các thông điệp có chứa dữ liệu.
a_data	D	8w	Dữ liệu tải (Data payload) cho các thông điệp có chứa dữ liệu.
a_corrupt	D	1	Dữ liệu trong nhịp (beat) này bị hỏng/lỗi.
a_valid	V	1	Bên gửi đang cung cấp tiến trình cho một hoạt động (Báo hiệu dữ liệu hợp lệ).

a_ready	R	1	Bên nhận đã chấp nhận tiến trình được cung cấp (Báo hiệu sẵn sàng nhận).

2.3.2. Kênh D – Response Channel

Kênh D mang phản hồi từ Slave về Master. Mỗi request trên kênh A luôn có đúng một response trên kênh D trả về do đó sẽ không tồn tại response không mang bất cứ thông tin gì.

Bảng 5. Các tín hiệu kênh D

Tín hiệu	Loại	Độ rộng	Mô tả
d_opcode	C	3	Mã lệnh (Operation code). Xác định loại thông điệp được truyền tải bởi kênh.
d_param	C	2	Mã tham số. Ý nghĩa phụ thuộc vào d_opcode; chỉ định quyền hạn cần chuyển giao hoặc mã phụ.
d_size	C	z	Logarit cơ số 2 của kích thước hoạt động: 2^n bytes.
d_source	C	o	Định danh nguồn Master (Master source identifier) trên mỗi liên kết.
d_sink	C	i	Định danh đích Slave (Slave sink identifier) trên mỗi liên kết.
d_denied	C	1	Slave không thể phục vụ yêu cầu này (Từ chối truy cập).
d_data	D	8w	Dữ liệu tải (Data payload) cho các thông điệp có chứa dữ liệu.
d_corrupt	D	1	Phát hiện lỗi/hỏng hóc trong phần dữ liệu tải.
d_valid	V	1	Bên gửi đang cung cấp tiến trình cho một hoạt động (Báo hiệu dữ liệu hợp lệ).
d_ready	R	1	Bên nhận đã chấp nhận tiến trình được cung cấp (Báo hiệu sẵn sàng nhận).

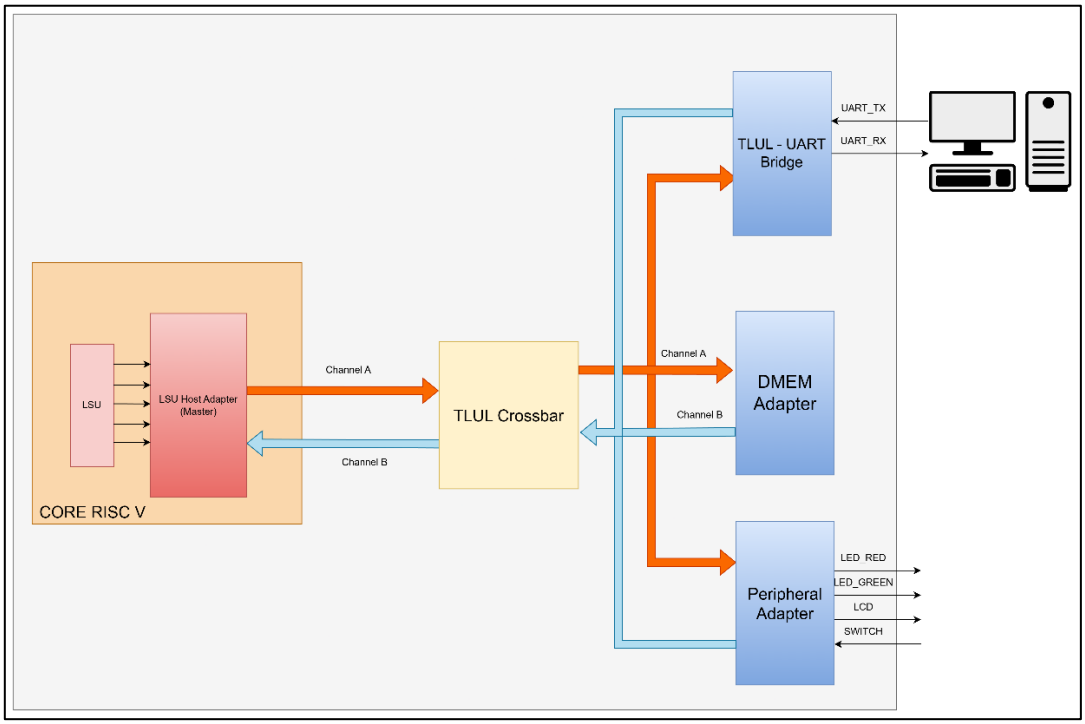
2.4. Ordering và Outstanding Transaction

TL-UL cho phép nhiều giao dịch outstanding, khi đó response có thể trả về không theo thứ tự gửi, khi đó *source* dùng để ghép response với request tương ứng. Để đáp ứng được điều này, Slave phải đảm bảo các thao tác trên cùng địa chỉ được tuần tự hóa hợp lệ, Master phải xử lý đúng response dựa trên *source*. Điều này giúp TL-UL tăng được throughput, cho phép CPU pipeline LSU hiệu quả hơn.

III. Phương pháp thiết kế

1. Tổng quan thiết kế

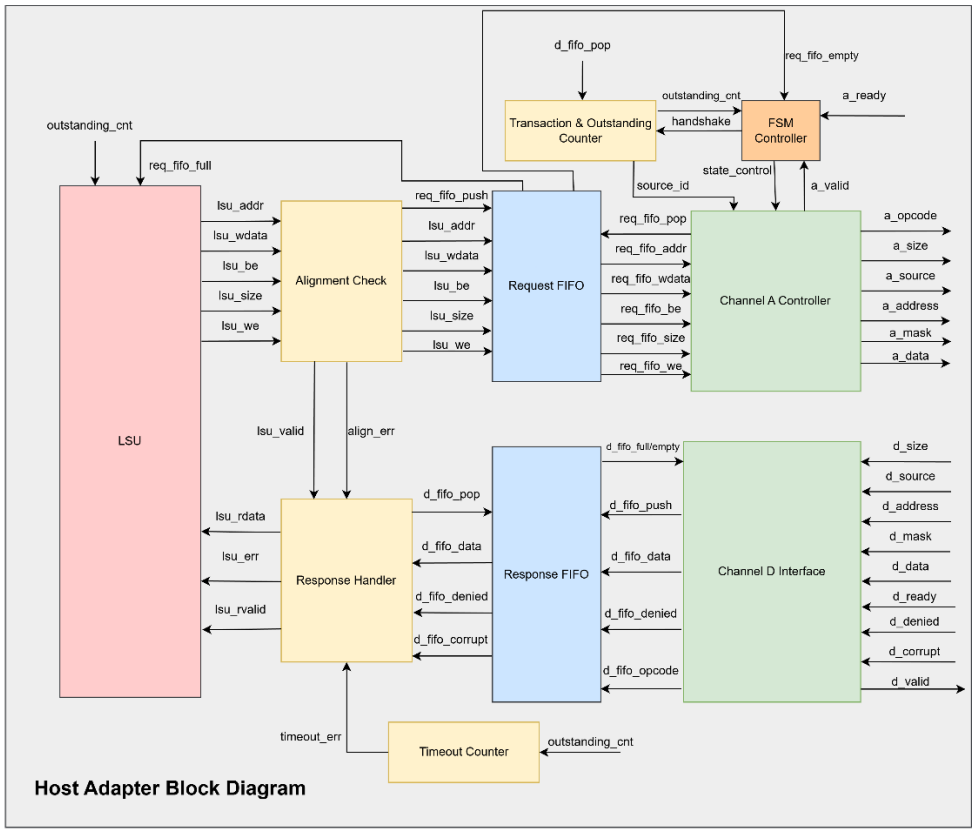
Hệ thống được thiết kế theo mô hình SoC đơn giản dựa trên giao thức TileLink Uncached Lightweight (TL-UL), trong đó CPU RISC-V đóng vai trò Master duy nhất, còn các khối bộ nhớ và ngoại vi đóng vai trò Slave. Mục tiêu của thiết kế là xây dựng một kiến trúc dễ mở rộng, dễ kiểm chứng và phù hợp cho MMIO, đồng thời giữ mức độ phức tạp phần cứng ở mức thấp.



Hình 1. Sơ đồ khối tổng quát thiết kế

Trong đồ án này, theo sơ đồ khối sẽ có 1 Master là core RISC-V được đặt cùng với Host Adapter với mục đích chuyển đổi tín hiệu của LSU theo chuẩn TL-UL, ở giữa sẽ có Crossbar với chức năng định tuyến dữ liệu đi và phân luồng các dữ liệu trả về từ kênh D của Slave và cuối cùng sẽ là 3 Slave bao gồm Memory, các ngoại vi Peripherrals và UART.

2. Thiết kế Host Adapter (LSU Adapter)



Hình 2. Sơ đồ khối Host Adapter

Trong sơ đồ khối, Host Adapter được chia thành các khối chức năng chính: Alignment Check, Request/Response FIFO, FSM điều khiển Channel A, Outstanding Transaction Counter, Channel D Interface và Response Handler.

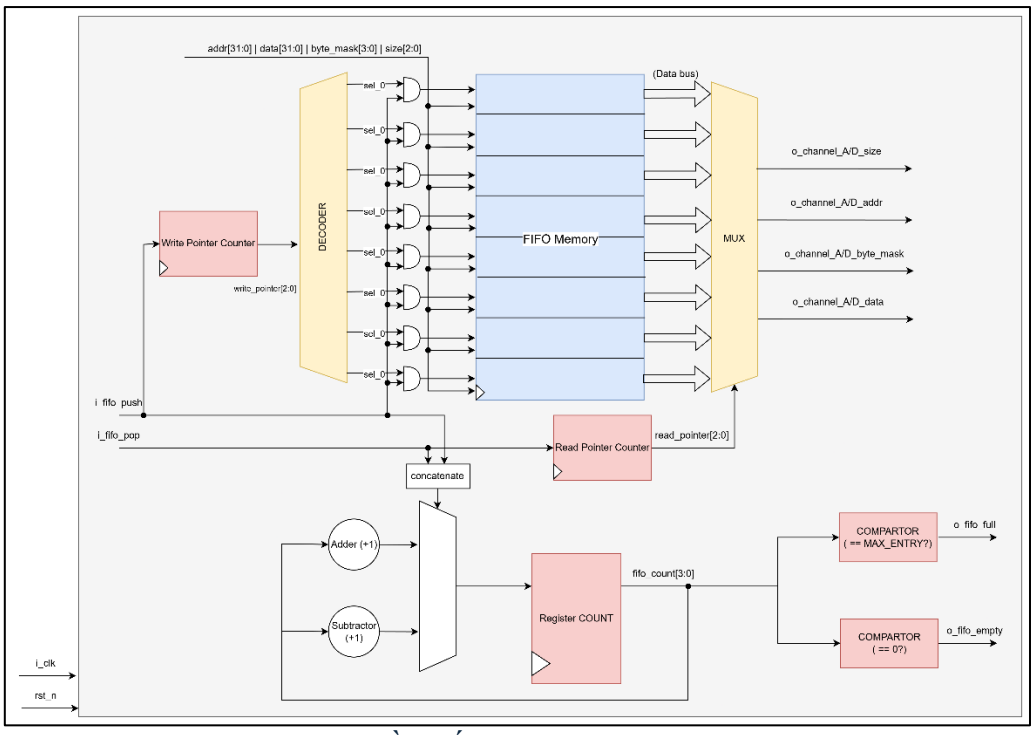
2.1. Alignment Check – Kiểm tra địa chỉ

Trước khi một yêu cầu từ LSU được chấp nhận, Host Adapter thực hiện kiểm tra căn chỉnh địa chỉ dựa trên kích thước truy cập (i_lsu_size):

- Byte (1B): không yêu cầu align.
- Halfword (2B): địa chỉ phải là chẵn.
- Word (4B): địa chỉ phải chi hết cho 4 (2 bit LSB bằng 0).

Ngoài ra, thiết kế chủ động loại bỏ các kích thước truy cập không hợp lệ đối với TLUL (size > 4 byte). Nếu khối này nhận địa chỉ từ CPU và phát hiện ra lỗi align thì request sẽ không được đẩy vào request FIFO, host adapter lập tức trả lỗi về LSU từ đó tránh phát sinh các transaction sai và không cần thiết trên bus. Các này giúp giảm gánh nặng xử lý lỗi cho Slave, đơn giản hóa các logic cho bus.

2.2. Request/Response FIFO



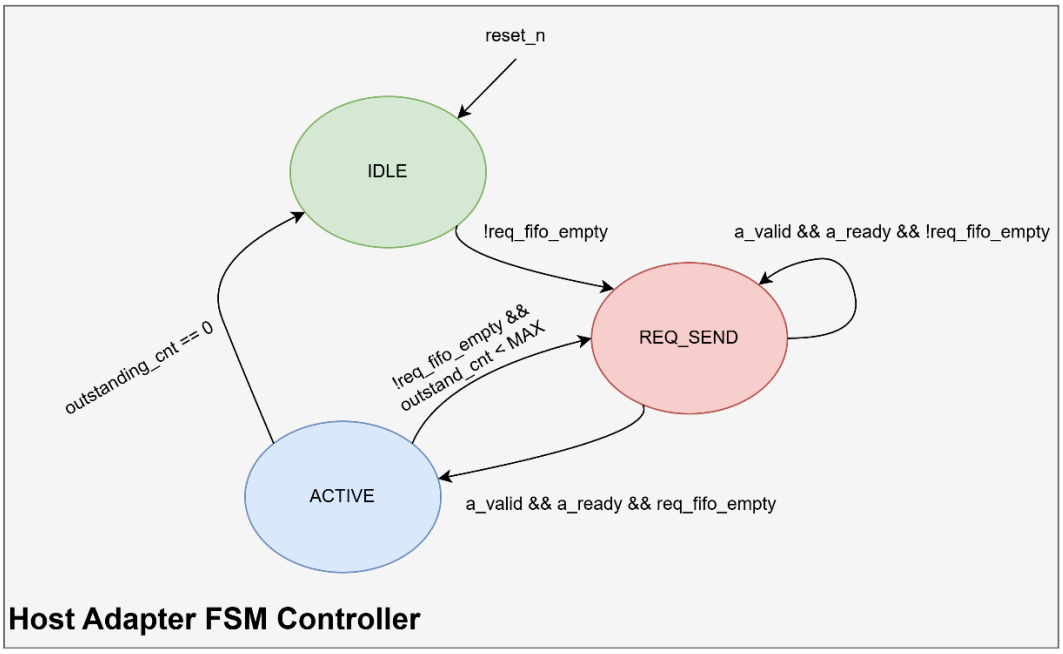
Hình 3. Sơ đồ khối Request/Response FIFO

Trong Host Adapter, hệ thống Request FIFO và Response FIFO được sử dụng như một cơ chế đệm trung tâm nhằm tách rời pipeline của CPU khỏi độ trễ không xác định của bus TL-UL và các Slave.

Request FIFO lưu trữ các yêu cầu load/store đến từ LSU sau khi đã vượt qua kiểm tra căn chỉnh địa chỉ, cho phép CPU phát sinh nhiều yêu cầu liên tiếp mà không phải chờ từng giao dịch hoàn tất trên bus.

Ngược lại, Response FIFO dùng để thu thập các phản hồi từ kênh D của TL-UL, bao gồm dữ liệu đọc và các cờ trạng thái như denied và corrupt, trước khi chuyển kết quả về LSU. Hai FIFO này phối hợp với nhau thông qua bộ đếm outstanding transaction: Request FIFO cho phép phát sinh song song nhiều giao dịch, trong khi Response FIFO đảm bảo mỗi giao dịch đều được xử lý và trả kết quả chính xác, ngay cả khi các phản hồi về không theo thứ tự. Ngoài ra, thiết kế còn hỗ trợ cơ chế bypass khi phản hồi từ Slave đến ngay trong cùng chu kỳ, giúp giảm độ trễ cho các thiết bị phản hồi nhanh như RAM hoặc thanh ghi MMIO. Việc sử dụng FIFO hai chiều giúp Host Adapter đạt được tính non-blocking, tăng throughput, đồng thời giữ cho logic điều khiển đơn giản, dễ kiểm chứng và phù hợp với các hệ thống SoC sử dụng TileLink Uncached Lightweight.

2.3. FSM Controller



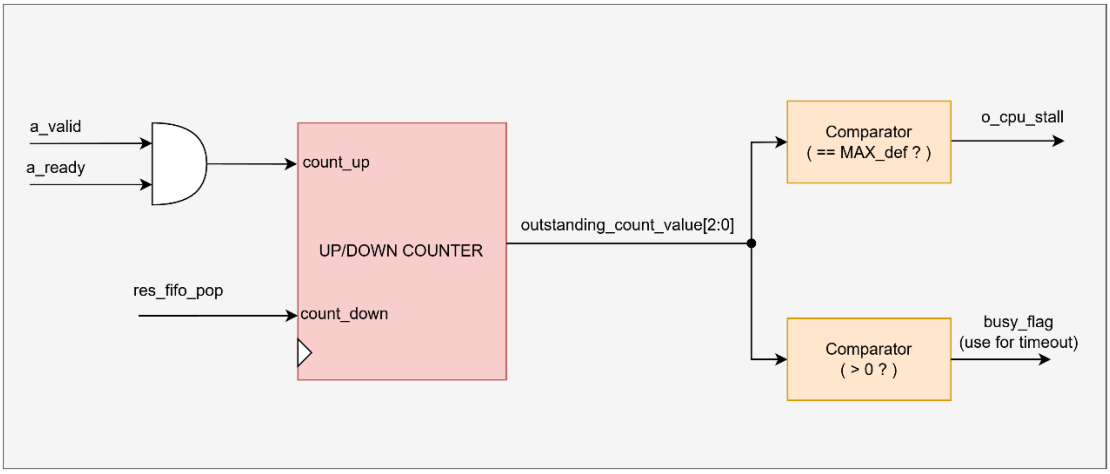
Hình 4. Thiết kế FSM

Host Adapter sử dụng một FSM đơn giản gồm 3 trạng thái:

- IDLE: không có request.
- REQ_SEND: đang phát request lên Channel A.
- ACTIVE: chờ response hoặc gửi tiếp request mới.

FSM đảm bảo request chỉ được gửi khi 2 tín hiệu a_valid và a_ready được handshake với nhau, đảm bảo không vượt quá số lượng outstanding transaction cho phép và có thể gửi liên tiếp nhiều request nếu FIFO còn dữ liệu. FSM này giúp thiết kế tránh stall không cần thiết, đảm bảo tuân thủ handshake TL-UL và dễ mở rộng cho nhiều Master trong tương lai.

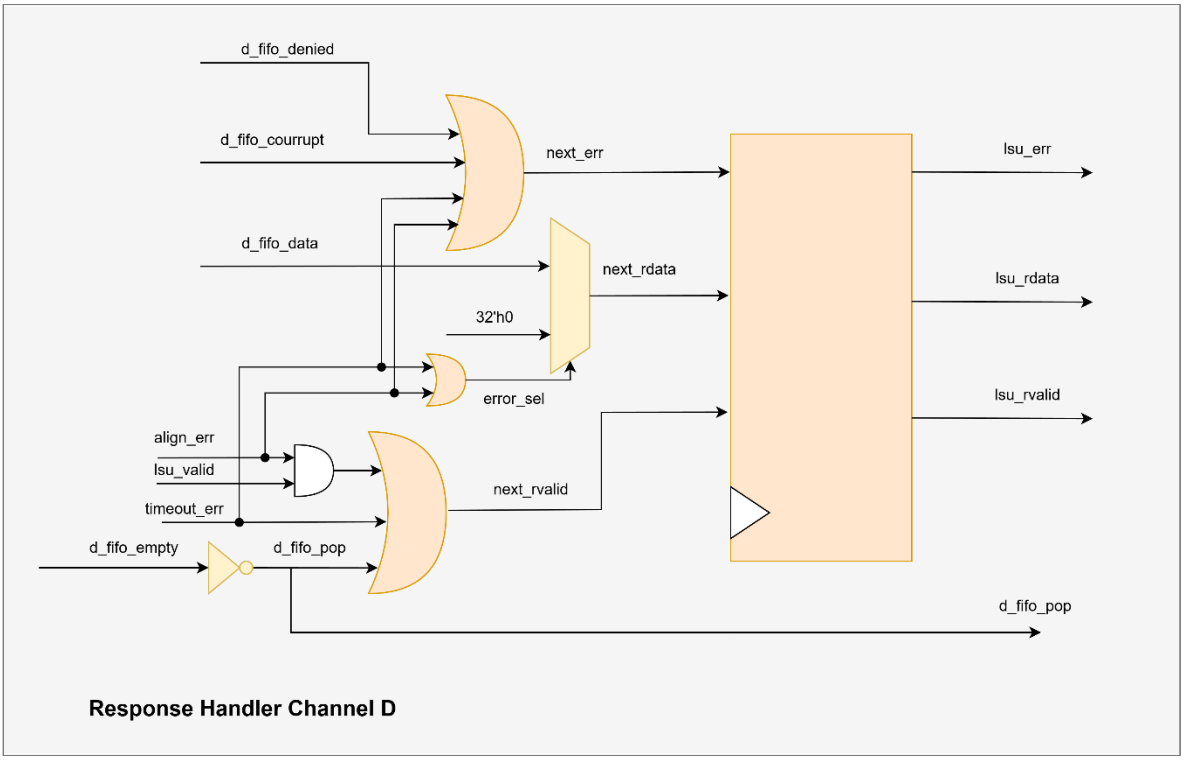
2.4. Outstanding Transaction Counter



Hình 5. Sơ đồ khối bộ Outstanding Counter

Khối Outstanding Counter giúp theo dõi số lượng request đã gửi nhưng chưa nhận được response. Khối này tăng khi request được handshake thành công trên Channel A và giảm khi response được pop khỏi Response FIFO để trả về CPU. Cơ chế này sẽ giới hạn số transaction đang treo, tránh làm quá tải các Slave và giúp FSM quyết định khi nào được phép gửi thêm request.

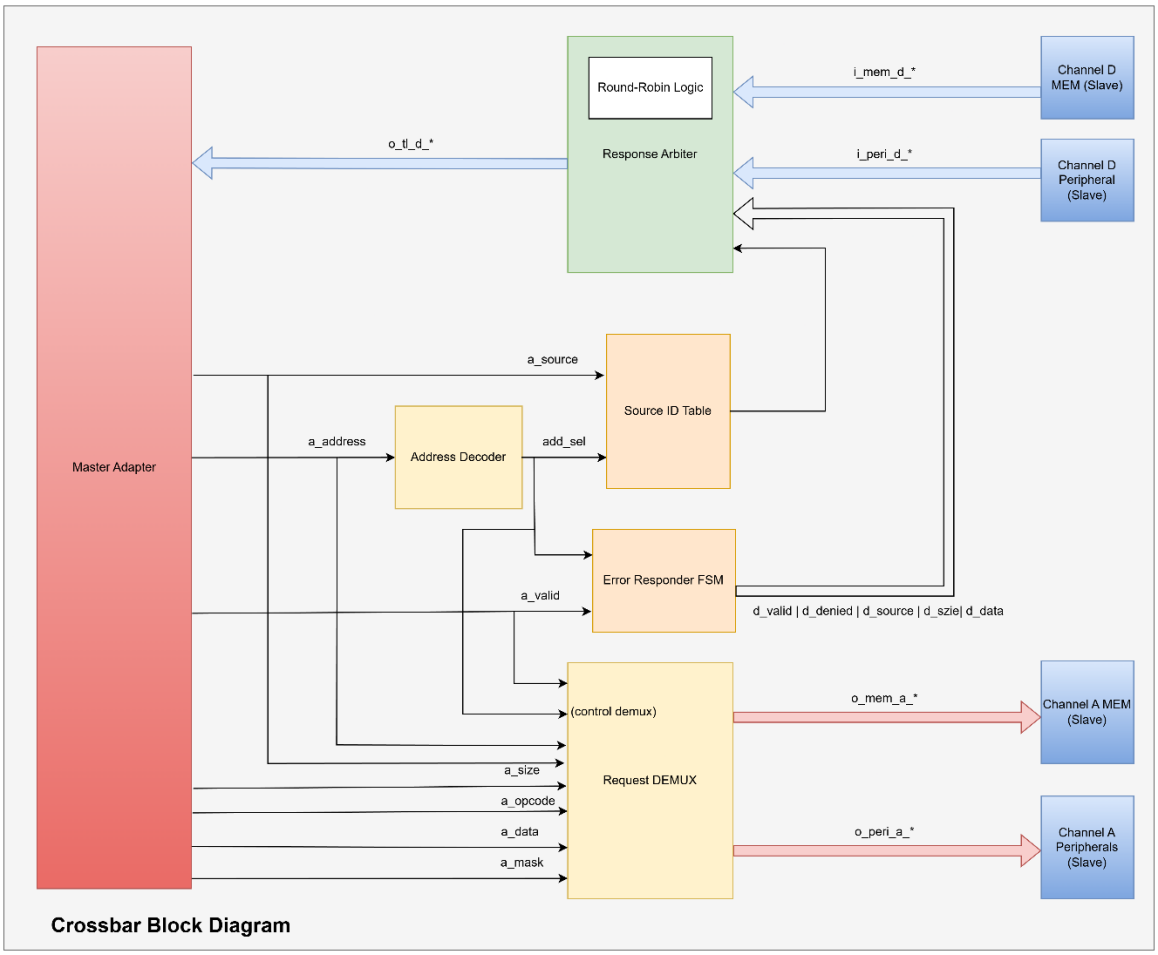
2.5. Response Handler – Trả kết quả cho LSU



Hình 6. Sơ đồ khối bộ Response Handler

Response Handler có tác dụng ghép response ứng với request dựa trên source ID, phát tín hiệu lsu_rvalid trong 1 chu kỳ để báo hiệu cho LSU biết có response trả về. Kết quả trả về LSU sẽ được trả về là dữ liệu đọc được từ Slave hoặc sẽ báo lỗi nếu xuất hiện các tín hiệu trả về như align error, timeout, access denied/corrupt data (từ channel D).

3. Thiết kế TL-UL Crossbar



Hình 7. Sơ đồ khối tổng quát TL-UL Crossbar

TL-UL Crossbar được thiết kế nhằm kết nối một Master duy nhất (Host Adapter của CPU) tới nhiều Slave khác nhau, bao gồm bộ nhớ (Memory), khối ngoại vi (Peripherals) và UART, thông qua chuẩn TileLink Uncached Lightweight. Crossbar chịu trách nhiệm định tuyến các yêu cầu từ Master đến đúng Slave dựa trên địa chỉ truy cập, đồng thời thu thập và phân phối các phản hồi từ các Slave về lại Master một cách chính xác và công bằng. Thiết kế này cho phép hệ thống mở rộng thêm Master và Slave trong tương lai mà không ảnh hưởng đến kiến trúc tổng thể của core RISC-V.

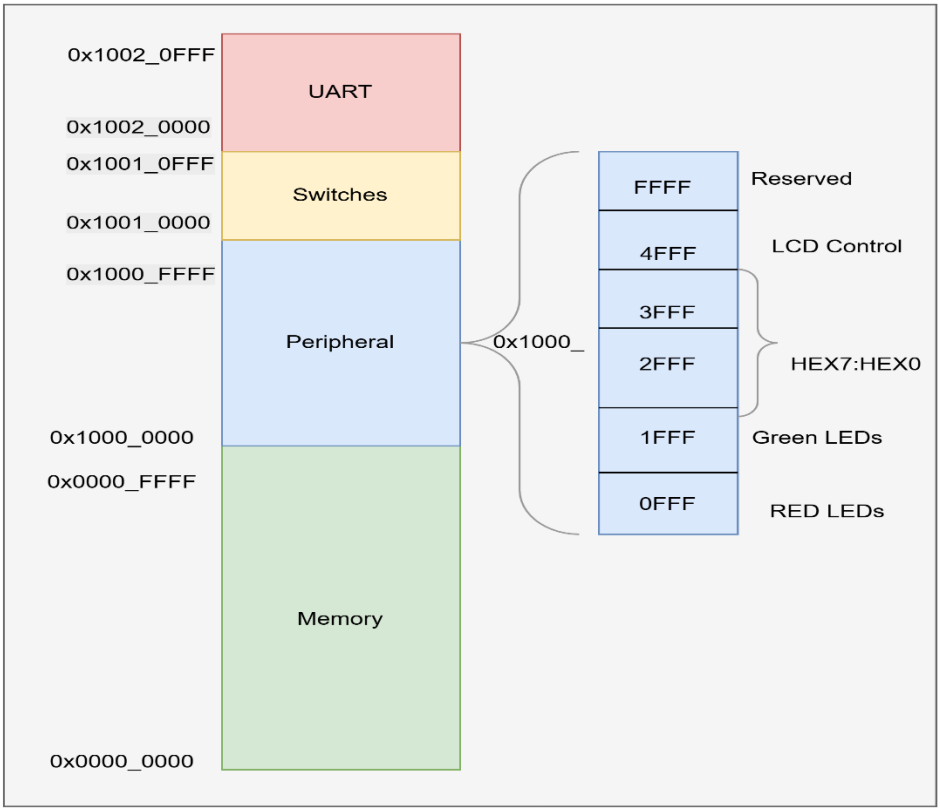
Crossbar được tổ chức thành các khối chức năng chính như sau:

- Address Decoder: xác định Slave đích của mỗi yêu cầu Channel A

- Request Demultiplexer (Request DEMUX): phân phối tín hiệu Channel A đến Slave tương ứng
- Source ID Table: lưu ánh xạ giữa source ID và Slave đích
- Error Responder FSM: tạo phản hồi lỗi cho các địa chỉ không hợp lệ
- Response Arbiter với Round-Robin Logic: chọn phản hồi Channel D từ các Slave
- Channel D Multiplexer: hợp nhất phản hồi về Master

Cách tổ chức này đảm bảo crossbar vừa tuân thủ giao thức TL-UL, vừa đơn giản, dễ mở rộng và dễ kiểm chứng

3.1. Address Decoder



Hình 8. Mapping địa chỉ các Slave

Crossbar sử dụng logic giải mã địa chỉ thuần tổ hợp để xác định Slave đích (addr_sel) cho mỗi transaction Channel A. Không gian địa chỉ được phân vùng rõ ràng:

- Memory: 0x0000_0000 – 0x0000_FFFF (64 KB).
- Peripheral IO (mỗi khối 4 KB): LED, HEX, LCD, Switch trong dải 0x1000_0000 – 0x1001_0FFF.
- UART: 0x1002_0000 – 0x1002_00FF.
- Error Region: Các địa chỉ không thuộc các vùng trên.

Nếu địa chỉ không khớp với bất kỳ Slave hợp lệ nào, yêu cầu sẽ được chuyển sang nhánh Error Response thay vì phát ra bus.

3.2. Request DEMUX – Định tuyến Channel A

Sau khi giải mã địa chỉ, Crossbar sử **dụng Request DEMUX** để định tuyến toàn bộ tín hiệu Channel A *(opcode, param, size, source, address, mask, data, corrupt)* đến đúng Slave tương ứng:

- SEL_MEM → Channel A của Memory
- SEL_PERI → Channel A của Peripheral
- SEL_UART → Channel A của UART

Crossbar không sửa đổi nội dung gói tin TL-UL, mà chỉ đóng vai trò chuyển tiếp (pass-through), đảm bảo tính trong suốt của giao thức. Tín hiệu o_tl_a_ready được phản hồi trực tiếp từ Slave tương ứng, giúp Master chỉ gửi request khi Slave sẵn sàng, đúng theo handshake của TileLink.

3.3. Source ID Table

Do TL-UL cho phép nhiều transaction outstanding, Crossbar cần ghi nhớ mỗi request đã được gửi tới Slave nào. Điều này được thực hiện thông qua Source ID Table, với chỉ số là a_source. Khi một request được handshake thành công (i_tl_a_valid && o_tl_a_ready), Crossbar lưu lại:

- source ID của request
- Slave đích tương ứng (SEL_MEM, SEL_PERI, SEL_UART)

Khi nhận phản hồi Channel D từ các Slave, Crossbar kiểm tra d_source để xác định phản hồi đó có đến từ đúng Slave đã được ghi nhận hay không, từ đó đảm bảo không xảy ra routing sai hoặc xung đột response.

3.4. Error Responder FSM

Đối với các yêu cầu truy cập vào vùng địa chỉ không hợp lệ, Crossbar kích hoạt Error Responder FSM. FSM này có hai trạng thái:

- ERR_IDLE: chờ yêu cầu lỗi.
- ERR_RESPOND: phát phản hồi Channel D với *d_denied* bằng 1.

FSM tạo ra phản hồi: AccessAck cho các yêu cầu ghi hoặc AccessAckData cho các yêu cầu đọc và tín hiệu *d_denied* bằng 1 để báo lỗi truy cập Cách xử lý này giúp hệ thống không bị treo khi truy cập sai địa chỉ mà vẫn giữ đúng hành vi của TL-UL.

3.5. Response Arbiter và Round-Robin Logic

Do có nhiều Slave có thể phản hồi cùng lúc, Crossbar sử dụng Response Arbiter để chọn một phản hồi duy nhất đưa về Master trên Channel D. Cơ chế phân xử sử dụng Round-Robin Priority, đảm bảo tính công bằng giữa các Slave:

- MEM → PERI → UART
- PERI → UART → MEM
- UART → MEM → PERI

Thứ tự ưu tiên được cập nhật sau mỗi handshake thành công trên Channel D *(o_tl_d_valid && i_tl_d_ready),* giúp tránh starvation cho bất kỳ Slave nào.

Error response luôn có độ ưu tiên cao nhất, đảm bảo phản hồi lỗi được xử lý kịp thời.

3.6. Channel D Routing về Master

Sau khi Arbiter chọn được phản hồi hợp lệ, Crossbar tiến hành chuyển tiếp toàn bộ tín hiệu Channel D về Master, kích hoạt *o_tl_d_valid* nhằm phản hồi cho *ready* của Slave đang được chọn. Thiết kế này đảm bảo chỉ 1 Slave được trả tín hiệu về trong mỗi chu kỳ và giữ được tính nhất quán giữa request và response.

4. Thiết kế các Slave Adapter

Trong kiến trúc TL-UL, mỗi Slave không trực tiếp xử lý giao thức TileLink mà thông qua một Slave Adapter làm lớp trung gian. Slave Adapter có nhiệm vụ chuyển đổi các transaction TL-UL từ Crossbar thành giao diện nội bộ phù hợp với từng loại Slave, đồng thời tạo phản hồi Channel D đúng chuẩn để trả về Master. Cách tiếp cận này giúp tách biệt rõ ràng giữa logic bus và logic chức năng, từ đó đơn giản hóa thiết kế, dễ mở rộng và thuận lợi cho việc kiểm chứng.

Trong hệ thống này, hai loại Slave Adapter chính được triển khai là:

- Memory Adapter cho bộ nhớ dữ liệu (DMEM)
- Peripheral Adapter cho các ngoại vi dạng memory-mapped I/O

4.1. Thiết kế TL-UL Memory Adapter

Memory Adapter đóng vai trò kết nối giữa bus TileLink Uncached Lightweight và khối bộ nhớ (BRAM), cho phép CPU thực hiện các phép load và store thông qua cơ chế truy cập bộ nhớ ánh xạ địa chỉ. Adapter tiếp nhận các yêu cầu từ kênh A của TL-UL, giải mã địa chỉ để xác định vị trí truy cập trong bộ nhớ, đồng thời hỗ trợ thao tác đọc và ghi với độ rộng dữ liệu 32-bit và mặt nạ byte (byte mask). Do DMEM có độ trễ xác

định và phản hồi nhanh, Memory Adapter được thiết kế theo mô hình phản hồi đơn giản: mỗi yêu cầu hợp lệ sẽ được xử lý tuần tự và phản hồi ngay trên kênh D với gói AccessAck hoặc AccessAckData tương ứng. Cách tiếp cận này giúp giảm độ phức tạp của logic điều khiển, đồng thời đảm bảo độ trễ thấp và tính quyết định cao cho các truy cập dữ liệu của CPU.

4.2. TL-UL Peripheral Adapter

Peripheral Adapter cung cấp giao diện kết nối TL-UL cho các ngoại vi đơn giản như LED, LCD và công tắc, thông qua cơ chế Memory-Mapped I/O. Tương tự như các adapter khác, khối này tiếp nhận yêu cầu từ kênh A, giải mã địa chỉ để ánh xạ tới từng thanh ghi điều khiển ngoại vi cụ thể. Các thao tác ghi được sử dụng để điều khiển trạng thái đầu ra (ví dụ bật/tắt LED hoặc ghi dữ liệu hiển thị), trong khi các thao tác đọc cho phép CPU lấy trạng thái của các tín hiệu đầu vào như công tắc. Do các ngoại vi này không yêu cầu băng thông cao và không phát sinh độ trễ lớn, Peripheral Adapter được thiết kế tối giản, không cần cơ chế pipeline hay FIFO, nhưng vẫn tuân thủ đầy đủ bắt tay và định dạng gói tin của TL-UL. Thiết kế này giúp hệ thống dễ mở rộng thêm ngoại vi mới mà không ảnh hưởng đến kiến trúc bus tổng thể.

5. Thiết kế UART và TL-UL UART Bridge

5.1. Thiết kế UART Transmitter (TX)

Khối UART Transmitter đảm nhiệm việc phát dữ liệu song song 8-bit ra đường nối tiếp uart_tx theo chuẩn UART. Dữ liệu cần truyền được đưa vào khối TX cùng với xung kích hoạt send_en, tại thời điểm này dữ liệu được chốt vào thanh ghi nội bộ để đảm bảo ổn định trong suốt quá trình truyền.

UART TX cũng sử dụng FSM điều khiển quá trình phát với các trạng thái IDLE, SENDING và DONE. Khi bắt đầu truyền, TX phát start bit ở mức thấp, sau đó lần lượt phát 8 bit dữ liệu từ LSB đến MSB, và kết thúc bằng stop bit ở mức cao. Bộ tạo baud rate đảm bảo mỗi bit được giữ ổn định trong đúng khoảng thời gian baud đã cấu hình. Trong suốt quá trình truyền, tín hiệu tx_busy được giữ ở mức cao để báo trạng thái bận, và tx_done được phát khi hoàn tất toàn bộ khung truyền. Thiết kế này cho phép CPU hoặc Bridge dễ dàng đồng bộ việc phát dữ liệu mà không cần can thiệp trong từng bit.

5.2. Thiết kế UART Receiver (RX)

Khối UART Receiver được thiết kế để chuyển đổi dữ liệu nối tiếp từ đường uart_rx thành dữ liệu song song 8-bit, hoạt động đồng bộ với xung clock hệ thống. Do tín hiệu UART là bất đồng bộ so với clock nội bộ, đầu vào uart_rx được đưa qua bộ đồng bộ hai tầng nhằm giảm nguy cơ metastability. Trên tín hiệu đã đồng bộ, cạnh xuống của start bit được phát hiện để kích hoạt quá trình nhận dữ liệu.

UART RX sử dụng Finite State Machine (FSM) gồm các trạng thái IDLE, RECEIVING và STOP. Sau khi phát hiện start bit, bộ tạo baud rate được kích hoạt để tạo xung lấy mẫu ở tốc độ cấu hình. Bit start được lấy mẫu tại trung tâm để xác nhận tính hợp lệ, sau đó các bit dữ liệu được lấy mẫu tuần tự tại mỗi chu kỳ baud. Dữ liệu được dịch vào thanh ghi dịch và được chốt lại sau khi nhận đủ 8 bit dữ liệu và stop bit hợp lệ. Khi một byte hoàn chỉnh được nhận, RX phát xung rx_done trong một chu kỳ và xuất dữ liệu ra data_byte, đồng thời tín hiệu rx_busy phản ánh trạng thái đang nhận dữ liệu của khối RX.

5.3. Thiết kế TL-UL UART Bridge

TL-UL UART Bridge là khối trung gian kết nối giữa bus TileLink Uncached Lightweight và các khối UART RX/TX, cho phép CPU truy cập UART thông qua cơ chế Memory-Mapped I/O (MMIO). Bridge tiếp nhận các yêu cầu đọc/ghi từ kênh A của TL-UL, giải mã địa chỉ để xác định thanh ghi UART tương ứng, sau đó điều khiển hoạt động của UART RX hoặc TX và trả phản hồi qua kênh D.

Bridge quản lý các thanh ghi nội bộ bao gồm thanh ghi dữ liệu truyền (TX Data), thanh ghi dữ liệu nhận (RX Data), cờ hợp lệ của RX, trạng thái bận của TX và thanh ghi cấu hình baud rate. Các yêu cầu ghi TL-UL được sử dụng để nạp dữ liệu truyền hoặc cấu hình UART, trong khi các yêu cầu đọc cho phép CPU kiểm tra trạng thái và lấy dữ liệu nhận. Để đơn giản hóa thiết kế, Bridge chỉ cho phép một giao dịch TL-UL đang chờ phản hồi tại một thời điểm, thông qua cơ chế pending response, đảm bảo tuân thủ chặt chẽ bắt tay của TL-UL mà không cần FIFO phức tạp.

Đối với dữ liệu nhận, Bridge lưu byte nhận được từ UART RX vào thanh ghi RX và set cờ hợp lệ, cờ này chỉ được clear khi CPU thực hiện thao tác đọc, giúp tránh mất dữ liệu khi CPU truy cập chậm. Đối với dữ liệu truyền, Bridge tạo xung kích hoạt để khởi động UART TX khi CPU ghi dữ liệu. Thiết kế UART Bridge vì vậy vừa đảm bảo tính

đúng đắn của giao thức TL-UL, vừa cung cấp một giao diện UART đơn giản, ổn định và dễ mở rộng trong hệ thống SoC.

IV. Kiểm tra và xác minh thiết kế

1. Mục tiêu kiểm tra và xác minh

Mục tiêu của quá trình kiểm tra và xác minh là đảm bảo toàn bộ hệ thống được thiết kế hoạt động đúng theo đặc tả, tuân thủ giao thức TileLink Uncached Lightweight và đáp ứng yêu cầu chức năng của CPU cũng như các ngoại vi. Công tác xác minh tập trung vào việc kiểm tra tính đúng đắn của luồng giao dịch TL-UL, khả năng xử lý song song các yêu cầu, cơ chế handshake giữa các khối, cũng như việc truyền nhận dữ liệu chính xác giữa CPU, bộ nhớ và các ngoại vi.

2. Kiểm tra và xác minh TL-UL Crossbar và các Slave Adapter

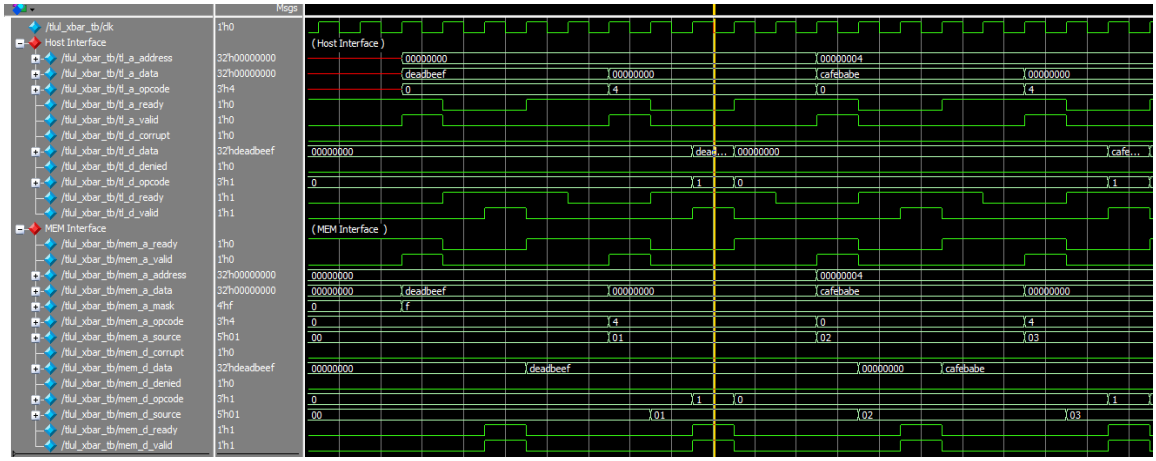
Để xác minh tính đúng đắn của thiết kế bus TileLink Uncached Lightweight và các khối Slave Adapter (Memory, Peripheral, UART), một testbench mức hệ thống đã được xây dựng. Testbench này mô phỏng toàn bộ luồng giao tiếp từ phía Host (CPU giả lập) thông qua TL-UL Crossbar đến các Slave, đồng thời kiểm tra phản hồi trả về trên kênh D.

Testbench hoạt động với xung nhịp 50 MHz và hỗ trợ cơ chế reset đồng bộ. Crossbar được cấu hình với ba Slave gồm bộ nhớ, khối ngoại vi và UART Bridge, phản ánh đúng kiến trúc hệ thống thực tế.

- Test 1: Memory Word Write/Read

Kết quả từ log thu được:

```
# [145000] WRITE: addr=0x00000000, data=0xdeadbeef
# [195000] WRITE OK: TL_D_ACCESS_ACK = 0
# [195000] READ: addr=0x00000000
# [245000] READ OK: TL_D_ACCESS_ACK_DATA = 1 | data=0xdeadbeef
# [245000] WRITE: addr=0x00000004, data=0xcafebab
```



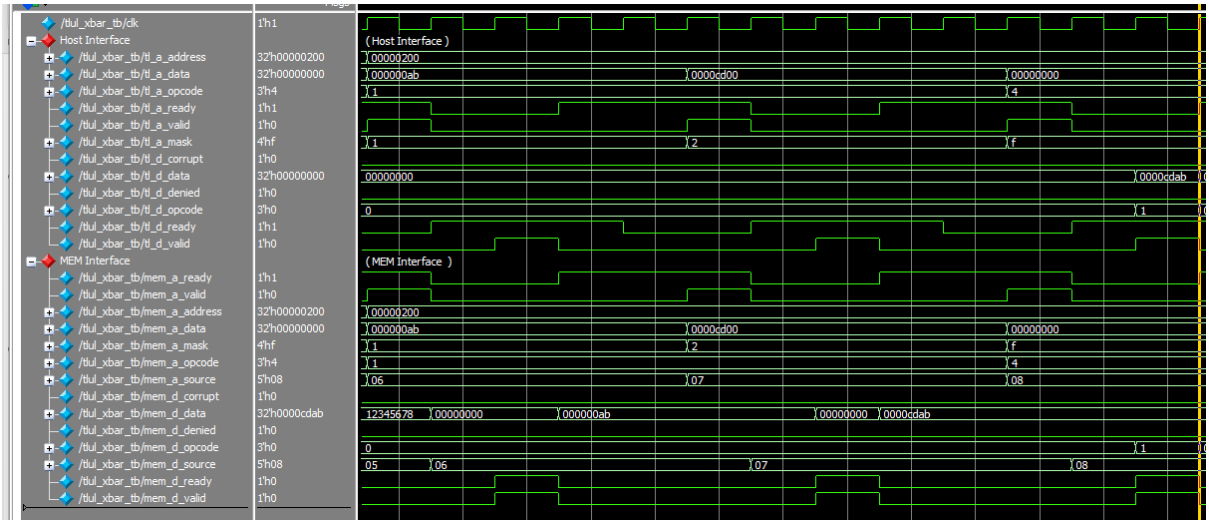
Hình 9. Waveform kiểm chứng tác vụ cơ bản của TL-UL

Nhận xét: Thấy rõ cặp handshake: *tl_a_valid = 1 gặp tl_a_ready = 1* trong 1 chu kỳ clock. Địa chỉ 0x00000000 xuất hiện ở cổng Host và truyền thông suốt đến cổng Memory (*mem_a_valid* bật lên). Dữ liệu đọc về (*tl_d_data*) khớp với dữ liệu đã ghi (0xDEADBEEF).

- Test 2: Byte Access

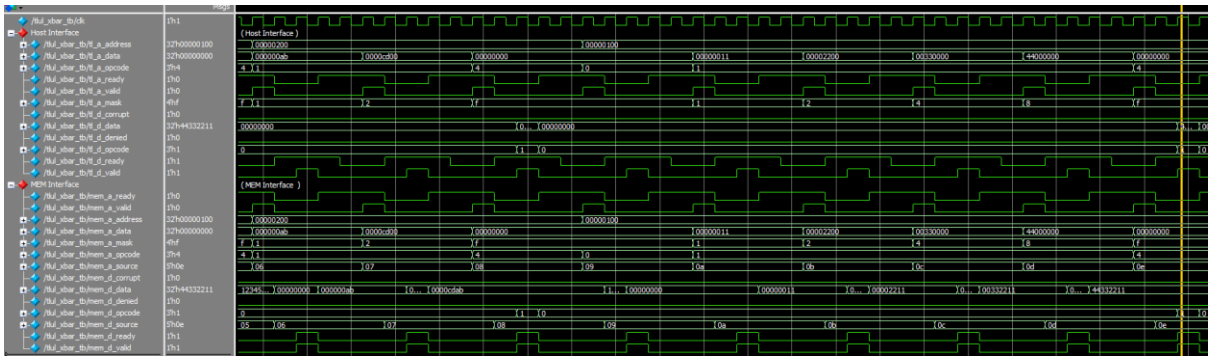
Kết quả từ log thu được:

```
# [445000] BYTE WRITE: addr=0x00000200, data=0xAB
# [495000] BYTE WRITE: addr=0x00000201, data=0xCD
# [545000] READ WORD: addr=0x00000200
# [595000] BYTE ACCESS OK: data=0x0000cdab
```

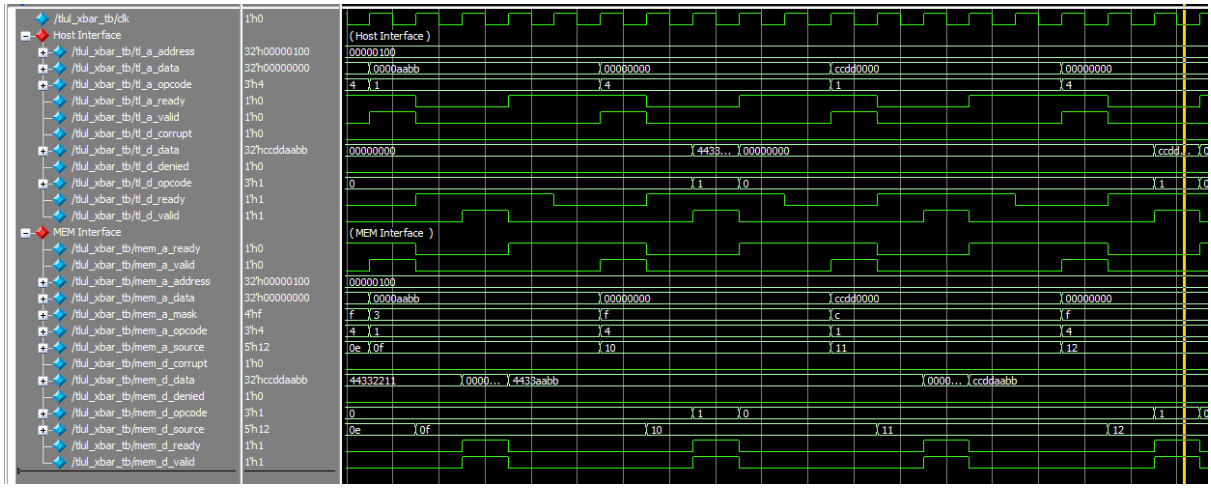


Hình 10. Waveform kiểm tra Byte Access

Nhận xét: Khi ghi byte, tl_a_mask chỉ bật 1 bit (ví dụ 4'b0001 hoặc 4'b0010).tl_a_opcode là PutPartialData. Kết quả đọc lại (tl_d_data) cho thấy byte cũ và byte mới được trộn đúng (ví dụ 0x0000CDAB).



Hình 11. Waveform kiểm tra khả năng ghi theo từng Byte

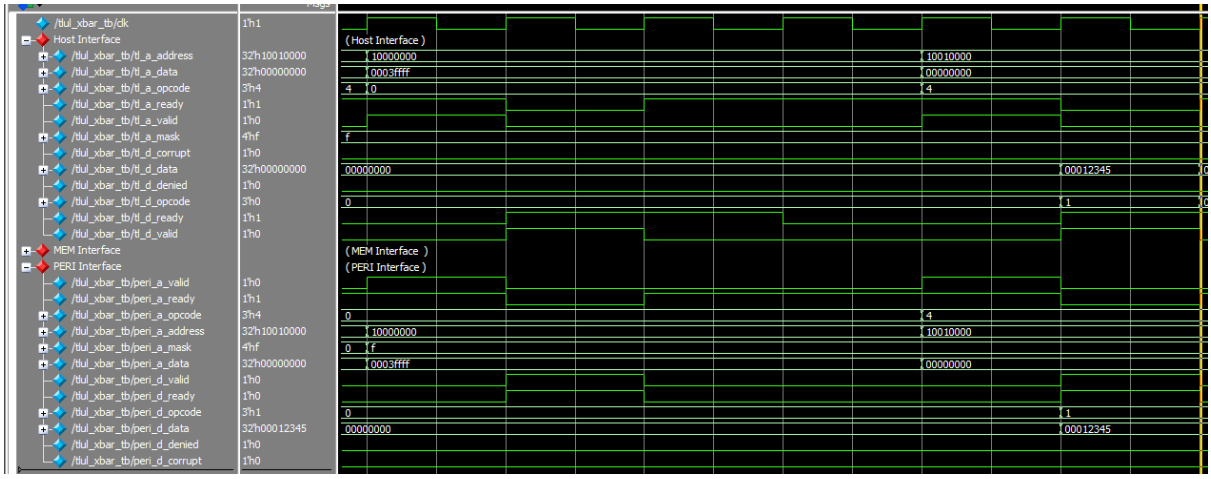


Hình 12. Waveform kiểm tra khả năng ghi Halfword

- Test 3: Peripheral access

Kết quả từ log thu được:

```
# [1095000] PERI WRITE: LEDR=0x0003ffff
# [1135000] LEDR OK: output=0x3ffff
# [1135000] PERI READ: Switch
# [1175000] SWITCH OK: data=0x00012345
```

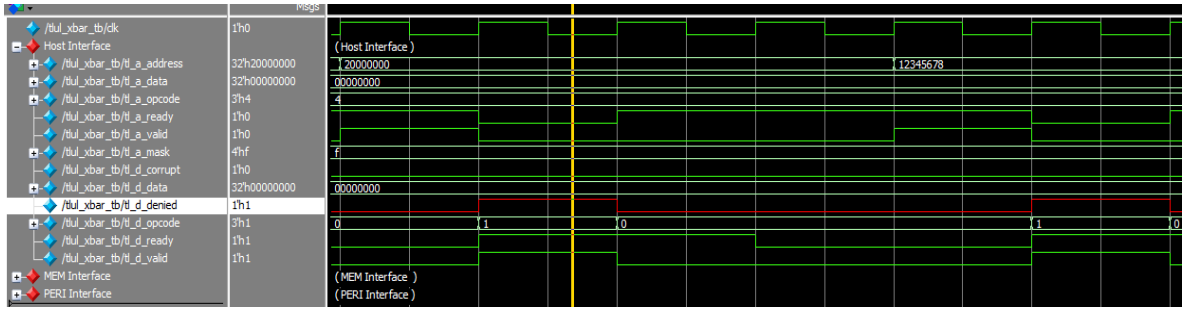


Hình 13. Waveform kiểm chứng khả năng ghi vào LED và đọc input từ Switch

Waveform ghi vào LED và đọc input từ Switch

Nhận xét: Hình này chứng minh Crossbar định tuyến đúng địa chỉ ngoại vi (Address Decoding) và điều khiển được phần cứng thực.

- Test 4: Invalid address

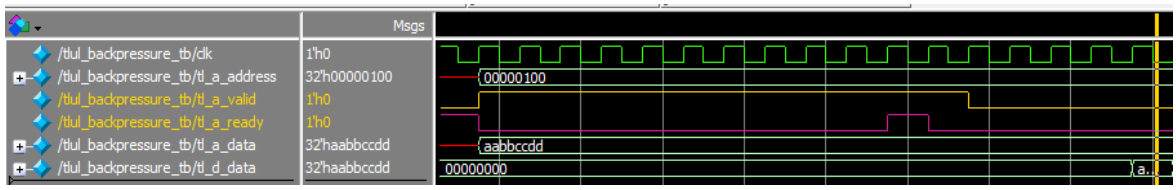


Hình 14. Waveform kiểm tra trường hợp access địa chỉ không được map

Nhận xét: ta thấy khi cố tình truy cập vào các địa chỉ không được mapping như 0x200000, 0x12345678 thì tín hiệu sẽ không truyền qua các Slave mà trực tiếp đưa phản hồi ra channel với *denied = 1* và *data = 0* đúng như mong muốn.

3. Kiểm tra xử lý backpressure và flow control

- Test 1: Slave Backpressure – khả năng giữ yêu cầu khi Slave bận.

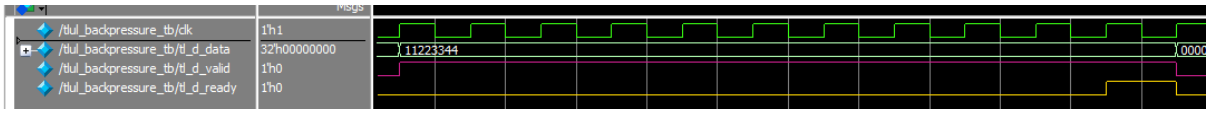


Hình 15. Waveform kiểm tra salve backpressure

Nhận xét: *tl_a_valid* lên 1 (Master muốn gửi), *tl_a_ready* bị ép xuống 0 (Slave bận) trong khoảng 5-10 chu kỳ clock. Khi đó *tl_a_valid* vẫn phải giữ ở mức 1 trong suốt thời

gian đó và không được phép rớt xuống cho đến khi *ready* lên 1. Ngay khi *tl_a_ready* lên 1 thì handshake xảy ra và khi đó *tl_a_valid* mới được xuống 0.

- Test 2: Host Backpressure – khả năng giữ phản hồi khi Master bận.

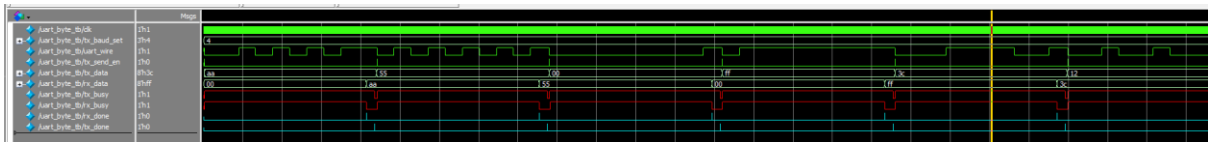


Hình 16. Waveform kiểm tra Master backpressure

Nhận xét: khi muốn Slave trả dữ liệu về *(tl_d_valid = 1)*, gặp Master đang bận *(tl_d_ready = 0)* thì Slave giữ nguyên *tl_d_valid* và *tl_d_data* cho đến khi Master rảnh *(tl_d_ready = 1)*.

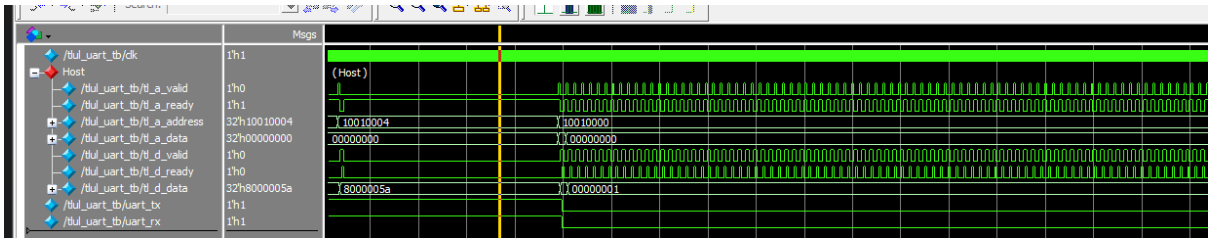
4. Kiểm tra và xác minh UART và TL-UL UART Bridge

- Test 1: Kiểm tra truyền nhận của UART



Hình 17. Waveform kiểm tra khả năng truyền nhận của UART

- Test 2: Kiểm tra truyền nhận vòng lặp (UART Loopback) qua UART Bridge



Hình 18. Waveform kiểm tra khả năng truyền nhận khi UART được gán với TL-UL

Nhận xét: Host ghi dữ liệu 0x5A vào thanh ghi TX (0x10010000), kích hoạt tín hiệu truyền nối tiếp trên chân *uart_tx*. Tín hiệu *uart_rx* thu lại dữ liệu đồng thời qua cơ chế loopback. Host đọc thanh ghi RX (0x10010004) và nhận về giá trị 0x8000005A. Dữ liệu đọc về khớp với dữ liệu gửi đi (0x5A) và cờ báo hiệu hợp lệ được bật, chứng tỏ khối UART hoạt động đúng chức năng và đảm bảo toàn vẹn dữ liệu.

5. Kiểm tra TL-UL bus khi tích hợp vào core RISC-V

- Test 1: Test ISA từ đó kiểm tra đọc/ghi của Memory

```
# SOC PIPELINE (TileLink) - ISA tests
#
# add......PASS
#
# addi.....PASS
#
# sub......PASS
#
# and......PASS
#
# andi.....PASS
#
# or.......PASS
#
# ori......PASS
#
# xor......PASS
#
# xori.....PASS
#
# slt......PASS
#
# slti.....PASS
#
# sltu.....PASS
#
# sltiu....PASS
#
# sll......PASS
#
# slli.....PASS
#
# srl......PASS
#
# srli.....PASS
#
# sra......PASS
#
# srai.....PASS
#
# lw.......PASS
#
# lh.......PASS
#
# lhu......PASS
#
# lb.......PASS
#
# lbu......PASS
```

```
lhu......PASS

lb.......PASS

lbu......PASS

sw.......PASS

sh.......PASS

sb.......PASS

auipc....PASS

lui......PASS

beq......PASS

bne......PASS

blt......PASS

bltu.....PASS

bge......PASS

bgeu.....PASS

jal......PASS

jalr.....PASS

malgn....ERROR

iosw.....PASS

=== END of ISA test ===

Statistics:
  Cycles:        9789
  Instructions:  4915
  Control Xfer:  1706
  Mispredicts:   5
  IPC:           0.502
  Mispred Rate:  0.29%
** Note: $finish    : ../
   Time: 19630 ns  Iterat
End time: 23:18:39 on Dec
Errors: 0, Warnings: 7
```

Trước đây, CPU truy cập bộ nhớ lý tưởng chỉ tốn 1 chu kỳ clock bằng cách kết nối trực tiếp (point-to-point) cho các ngoại vi. Khi tích hợp TileLink UL, mọi giao dịch phải tuân thủ cơ chế bắt tay (Handshake), tốn tối thiểu **2 chu kỳ** (1 cycle gửi Request kênh A + 1 cycle nhận Response kênh D). Do đó, tốc độ nạp lệnh (Fetch) bị giới hạn, kéo IPC về sát mức lý thuyết là 0.5 (1 lệnh / 2 chu kỳ).

Về misaligned, các bộ nhớ trước đây cho phép truy cập địa chỉ lệch (ví dụ: đọc 4-byte tại địa chỉ 0x...01) bằng cách cho Memory làm Dual Port Ram, khi đó sẽ trả về 2 giá trị dữ liệu tại địa chỉ hiện tại và địa chỉ kế tiếp, việc còn lại là LSU xử lý. Tuy nhiên, Adapter TL-UL buộc địa chỉ phải chia hết cho kích thước dữ liệu (Aligned). Khi bài

test thực hiện lệnh truy cập lệch, hệ thống Bus không hỗ trợ dẫn đến dữ liệu trả về bị sai hoặc báo lỗi. Việc xử lý misaligned sẽ trở nên phức tạp do độ rộng bus bằng đúng với giá trị dữ liệu trả về là 32 bit, do đó tra phải gửi 2 lần transaction làm CPU bị stall lâu hơn.

- Test 2: Truyền UART, RX nhận và trả về core RISC-V sau đó hiển thị lên led 7 đoạn

```
=== SOC UART 7-SEG TEST (WITH BAUD CHECK) ===
[390000] Reset Released

--- TEST 1: Normal Operation (115200 Baud) ---
  [10390000] TX -> SOC (Baud Cycles: 434): 0x31 ('1')
      Display: [ 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 ]

--- TEST 2: Wrong Baud Rate (9600 Baud) - Should Fail ---
  [205870000] TX -> SOC (Baud Cycles: 5208): 0x32 ('2')
      Display: [ 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 ]

--- TEST 3: Stress Test  - Should Pass ---
  [1451630000] TX -> SOC (Baud Cycles: 425): 0x33 ('3')
      Display: [ 0 | 0 | 0 | 1 | 0 | 0 | 0 | 3 ]

--- TEST 4: Stress Test- Should Pass ---
  [1645130000] TX -> SOC (Baud Cycles: 443): 0x41 ('A')
      Display: [ 0 | 0 | 1 | 0 | 0 | 0 | 3 | A ]
```

Hình 19. Kết quả kiểm tra UART truyền nhận trong SoC

Set uart có baud rate là 115200, ta thấy được khi truyền vào nếu sai baudrate thì sẽ nhận ký tự rác và không hiển thị được.

V. Đánh giá hiệu năng thiết kế

1. Mô phỏng kiểm tra throughput của bus

Để đánh giá hiệu quả của giao thức TL-UL, ta sử dụng các công thức tính toán lý thuyết và thực nghiệm như sau:

- Băng thông lý thuyết tối đa được tính dựa trên tần số hoạt động và độ rộng đường dữ liệu của bus:

$$BW_{max} = f_{clk} + W_{data}$$

Trong đó f_{clk} có giá trị là 50 MHz (Tần số xung nhịp lý thuyết của hệ thống), W_{data} = 4 bytes (Độ rộng dữ liệu 32-bit). Khi đó băng thông lý thuyết tối đa ta tính được là 200 MB/s.

- Băng thông thực tế sẽ được đo lường bằng tổng số byte truyền được trong khoảng thời gian mô phỏng hoặc thực tế:

$$BW_{actual} = \frac{\sum Số\ bytes\ truyền\ được}{Tổng\ số\ cycles + T_{period}}$$

Để kiểm tra hiệu năng, ta lập tra các kịch bản mô phỏng khác nhau và tính toán được hiệu năng của thiết kế trong mô phỏng theo bảng dưới đây:

Bảng 6. Kết quả tính toán throughput của bus

Kịch bản kiểm thử	Tổng số giao dịch	Tổng số chu kỳ	Băng thông thực tế (Throughput)	Hiệu suất Bus	Đánh giá
Sequential Write	100	510	39.22 MB/s	19.6%	Đạt hiệu năng cao nhất nhờ cơ chế posted-write.
Sequential Read	100	1010	19.80 MB/s	9.9%	Thấp hơn ghi do độ trễ quay vòng (round-trip latency).
Burst Write (16-beat)	96	1157	16.59 MB/s	4.2%	Giới hạn bởi Flow Control khi bộ đệm (FIFO) đầy.
Burst Read (16-beat)	96	1301	14.76 MB/s	3.7%	Giới hạn bởi tốc độ xử lý Request của Slave.
Random Access	100	1899	10.53 MB/s	5.3%	Hiệu năng thấp nhất do địa chỉ thay đổi ngẫu nhiên.

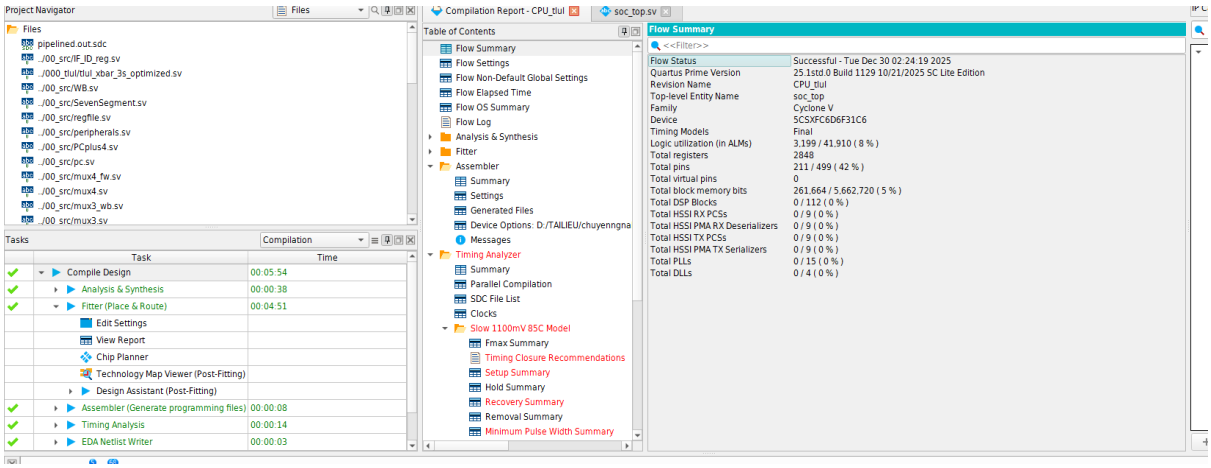
Có sự chênh lệch rõ rệt giữa hiệu năng ghi và đọc tuần tự. Tốc độ Ghi tuần tự đạt mức cao nhất (39.22 MB/s), trong khi tốc độ Đọc tuần tự chỉ đạt khoảng một nửa (19.80 MB/s). Nguyên nhân là do bản chất của thao tác Đọc đòi hỏi độ trễ quay vòng (round-trip latency): Master phải gửi yêu cầu, chờ Slave truy xuất dữ liệu từ bộ nhớ, và sau đó Slave mới trả dữ liệu về. Trong khi đó, thao tác Ghi có thể được Master "đẩy" đi (posted

write) mà không cần chờ dữ liệu phản hồi ngay lập tức để tiếp tục xử lý, giúp tận dụng băng thông hiệu quả hơn.

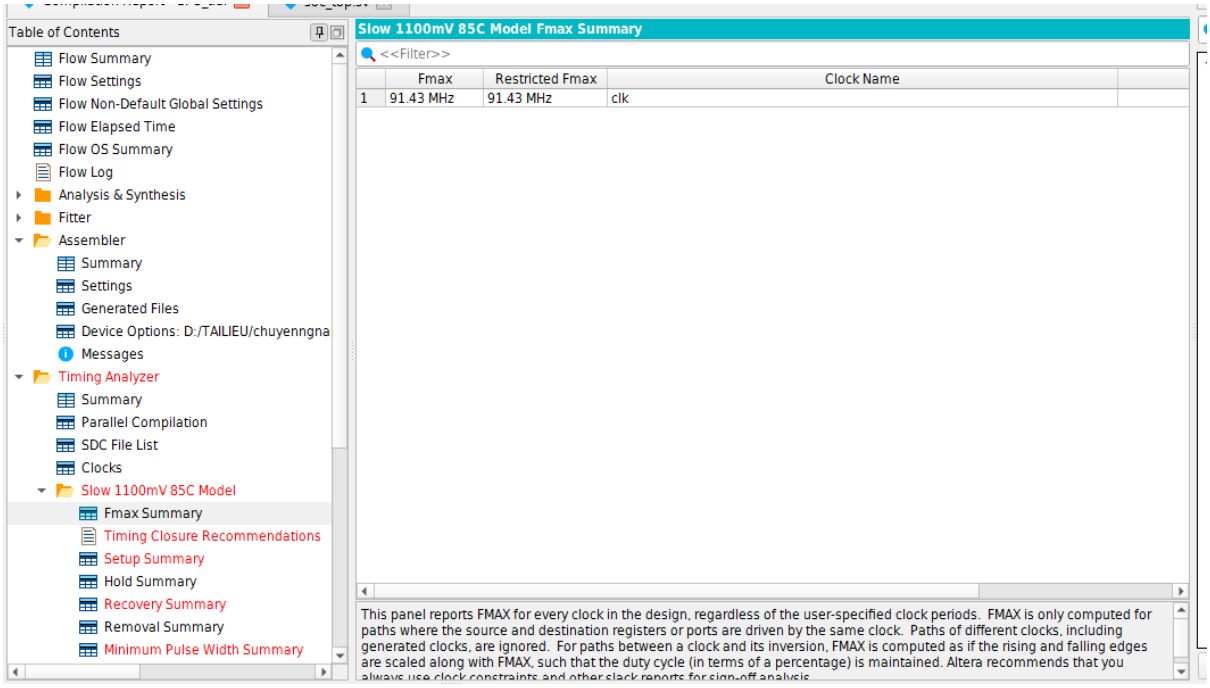
Một điểm đáng chú ý trong kết quả là hiệu năng ở chế độ Burst (truyền liên tiếp 16 giao dịch) lại thấp hơn so với chế độ Tuần tự đơn lẻ (~16.59 MB/s so với 39.22 MB/s). Sự sụt giảm này chỉ ra rằng hệ thống đang gặp giới hạn về Flow Control. Khi Master gửi các yêu cầu dồn dập, bộ đệm (FIFO) trong Adapter hoặc Crossbar nhanh chóng bị đầy, kích hoạt tín hiệu *backpressure* (hạ ready xuống thấp). Việc xử lý tín hiệu ngắt quãng này tạo ra các khoảng chết (bubbles) giữa các giao dịch, làm giảm thông lượng tổng thể. Mặc dù vậy, hệ thống vẫn đảm bảo tính toàn vẹn dữ liệu tuyệt đối trong điều kiện chịu tải cao.

Trong các kịch bản kiểm tra ngẫu nhiên (Random Access) và hỗn hợp (Mixed Read/Write), băng thông duy trì ổn định ở mức 10 - 12 MB/s. Đây là mức hiệu năng cơ sở chấp nhận được cho một hệ thống SoC đơn giản không sử dụng Cache. Kết quả kiểm tra khẳng định thiết kế Crossbar và Adapter hoạt động ổn định, không xảy ra tình trạng mất mát dữ liệu hay tắc nghẽn (deadlock) dưới mọi mô hình truy cập, đáp ứng tốt yêu cầu của một hệ thống vi điều khiển nhúng trên nền tảng FPGA.

2. Kết quả khi thực thi trên FPGA.



Hình 20. Tóm tắt tài nguyên sử dụng trên kit DE10



Hình 21. Tần số tối đa của thiết kế

Nhận xét: khi tích hợp thêm bus TL-UL vào core RISC-V để tạo ra được một SoC chuẩn, ta dễ dàng thấy được Fmax tăng lên 91MHz so với thiết kế pipeline 5 tầng có tần số tối đa là 50MHz sử dụng kết nối point-to-point với memory và các ngoại vi được tích hợp vào LSU ở MEM stage. Nguyên nhân có thể là do việc được bus ra ngoài làm giảm đường critical path trong MEM stage đặc biệt là LSU, nhờ đó thiết kế được cải thiện tần số cũng như được chuẩn hơn khi được so sánh với các thiết kế ngoài thực tế. Ngoài ra, thiết kế như thế này còn giúp chúng ta dễ dàng mở rộng thêm các Master hoặc Slave sau này.

VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Sau quá trình nghiên cứu và thực hiện, đồ án đã thiết kế và tích hợp thành công một hệ thống SoC hoàn chỉnh trên nền tảng FPGA, bao gồm lõi vi xử lý RISC-V 32-bit (RV32I) kiến trúc pipeline 5 tầng và hệ thống giao tiếp liên kết TileLink Uncached Lightweight (TL-UL). Về mặt kiến trúc, hệ thống đã xây dựng được các khối Host/Device Adapter và Crossbar định tuyến linh hoạt, đảm bảo tính toàn vẹn dữ liệu thông qua cơ chế bắt tay (handshake) chặt chẽ và xử lý tốt các trường hợp nghẽn mạch (backpressure). Về mặt hiệu năng thực nghiệm, hệ thống hoạt động ổn định ở tần số 50 MHz với chỉ số IPC đạt 0.502 và băng thông bộ nhớ đạt đỉnh 39.22 MB/s trong kịch bản ghi tuần tự. Các kết quả kiểm thử tích hợp (Integration Test) cho thấy sự phối hợp nhịp nhàng giữa phần cứng và phần mềm, minh chứng qua khả năng truyền nhận UART

chính xác và việc CPU thực thi luồng lệnh ổn định dù phải chịu chi phí độ trễ của giao thức bus thực tế.

Dựa trên các kết quả đo đạc hiệu năng, hướng phát triển tiếp theo của đề tài sẽ tập trung vào việc tối ưu hóa băng thông và giảm độ trễ hệ thống. Hệ thống bus cần được nâng cấp lên chuẩn TileLink Uncached Heavyweight (TL-UH) để hỗ trợ các giao dịch Burst thực thụ và bổ sung bộ điều khiển truy cập bộ nhớ trực tiếp (DMA) nhằm giảm tải cho CPU trong các tác vụ truyền dữ liệu khối lượng lớn. Cuối cùng, để tăng tính ứng dụng thực tế cho SoC, thiết kế sẽ được mở rộng thêm bộ điều khiển ngắt (PLIC/CLINT) và các chuẩn giao tiếp công nghiệp khác như SPI/I2C cao hơn thì sẽ là USB2.0, giúp hệ thống hoạt động hiệu quả hơn trong môi trường nhúng phức tạp thay vì chỉ dựa vào cơ chế vòng lặp chờ polling như hiện tại.

VII. TÀI LIỆU THAM KHẢO

[1] SiFive, Inc., *"TileLink Specification, Version 1.9.3"*, SiFive, 2023.
[2] lowRISC, *"OpenTitan Technical Specifications: TileLink UL Protocol"*, Opentitan, 2023.
[3] K. Asanović, R. Avizienis, J. Bachrach, et al., *"The Rocket Chip Generator"*, Technical Report UCB/EECS-2016-17, University of California, Berkeley, 2016
[4] Hai Cao, *"Milestone 3 — Computer Architecture: Design of Pipelined RISC-V Processors"*, HCMUT, 2024.